

The "re" Module

 <https://github.com/learn-co-curriculum/python-p3-re-module>  <https://github.com/learn-co-curriculum/python-p3-re-module/issues/new>

Learning Goals

- Validate that strings match specific patterns using regular expressions.
- Search for strings that match specific patterns using regular expressions.

Key Vocab

- **Regular Expression:** a sequence of characters used to search for a pattern inside of a string.
- **Pattern:** a description of sequences of characters that share certain traits with one another. Sequences do not need to be the same length or share any common characters to pattern match. Also called a **filter**.

Introduction

To start working with regular expressions in your IDE, you will first need to import the `re` module from Python's standard library. Python's standard library is a collection of modules that are downloaded when you install any version of Python. These modules aren't keyword objects like `print()` or `return`, but they can be imported without downloading them into your `pipenv`.

Open up your Python shell to get started:

```
import re
text = "This is some regular text."
```

If we go to [regex101](https://regex101.com/), we can try out some patterns to start thinking about how to craft our regular expression. If we use the text itself, we *do* get a match- the `.` character matches anything. If we want to match the period specifically, we should end our pattern

with `\. :`

```
import re
text = "This is some regular text."
pattern = r'This is some regular text\.'
```

Now let's dive into the `re` module to put this pattern to work.

re Methods

The first thing that we need to do is turn our pattern over to the `re` module. We'll use the `compile()` instance method:

```
import re
text = "This is some regular text."
pattern = r'This is some regular text\.'
regex = re.compile(pattern)
```

NOTE: You can use the `re` module without compiling your patterns beforehand. This is *not* recommended because it will require you to include your pattern as an argument to every new `re` method that you run. Gross!

Let's use Python's `dir()` function to explore our `regex` object:

```
dir(regex)
# => ['__class__', '__copy__', '__deepcopy__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
```

As with most Python objects, we can see that there are a lot of magic methods in there- we're typically not meant to call those, so we'll skip ahead to the standard instance methods. Let's start with `search()`.

search() , match() , and fullmatch()

The `search()` instance method of compiled `re` objects is exactly what it sounds like- it searches to see if there is a match for your regular expression in the text.

Let's take a look:

```
import re
text = "I love text. Text text text text text."
pattern = r'text'
regex = re.compile(pattern)
match = regex.search(text)
print(match)
# => <re.Match object; span=(7, 11), match='text'>
```

The `search()` method returns an `re.Match` object. Let's use `dir()` once again to learn more:

```
dir(match)
# => [(ignoring magic methods...), 'end', 'endpos', 'expand', 'group', 'groupdict', 'groups', 'lastgroup', 'lastindex'
```



An `re.Match` object contains an index for its start and end locations in the string that your RegEx is being checked against. It also contains the original string itself:

```
match.start()
# => 7
match.end()
# => 11
match.span()
# => (7, 11)
match.string
# => 'I love text. Text text text text text.'
```

There is also a `match()` method that belongs to `re` objects. The `match()` method searches to see if there is a match for your regular expression that starts from the *beginning* of the text, returning an `re.Match` object if one is found.

If you're feeling **very particular**, the `fullmatch()` method checks the text from beginning to end, returning an `re.Match` object if the whole thing is a match.

The `search()`, `match()`, and `fullmatch()` methods are useful if you're just checking to see if a match exists; if you're looking for the strings that match your pattern, `findall()` is a better option.

► Which `re` method returns an `re.Match` object for the first match anywhere in a string?

findall()

The `findall()` method is very straightforward (and therefore very useful). Like `search()`, it searches to see if there are matches for your RegEx within the text. `findall()` returns a list of matching strings instead of an `re.Match` object.

Let's use the `findall()` method to isolate the three letter words from a sentence:

```
import re
text = 'The big red cat ate the fat rat.'
pattern = r'[A-z]{3}'
regex = re.compile(pattern)
regex.findall(text)
# => ['The', 'big', 'red', 'cat', 'ate', 'the', 'fat', 'rat']
```

As `findall()` finds and returns all strings that match your RegEx (and we don't have to fuss around with `dir()` to find more information), it is the `re` method that you will use the most frequently as a software engineer.

split() and sub()

Regular expressions are helpful in finding strings that match certain patterns, but they also give us the ability to manipulate strings. The `re` module provides us many methods to do this- `split()` and `sub()` are the most useful.

`split()` returns a list of strings that surround a pattern that you choose to split around. Let's use `split()` to try and build a clear narrative for a 4-year-old's day at pre-K:

```
story = "I went to the park and I saw my friend and my friend's dog was there and we ran around and there was another  
and_pattern = re.compile(r'\sand')  
and_pattern.split(story)  
# => ['I went to the park', ' I saw my friend', ' my friend's dog was there', ' we ran around', ' there was another dc
```

► What does `\s` replace?

Now it's a bit easier to make sense of what happened there.

The `sub()` method allows us to further manipulate our search string. In addition to the search string, `sub()` takes a substitution as a parameter. Let's replace those "and"s with some punctuation:

```
and_pattern.sub(".", story)  
# => "I went to the park. I saw my friend. my friend's dog was there. we ran around. there was another dog. the other
```

That's much better.

Conclusion

The `re` module in Python's standard library allows us to use regular expressions to search, validate, and replace. `re` contains several methods that will help you accomplish string-related tasks:

- `search()`, `match()`, and `fullmatch()` check for a single match in your search string, returning `re.Match` objects if a match is found.
- `findall()` returns a list of matching strings.

- `split()` and `sub()` allow you to modify strings that would be more useful in a different format.
-

Resources

- [re - Regular expression operations - Python](https://docs.python.org/3/library/re.html)  [\(https://docs.python.org/3/library/re.html\)](https://docs.python.org/3/library/re.html)
- [regex101](https://regex101.com/)  [\(https://regex101.com/\)](https://regex101.com/)
- [Python Regular Expressions - Google for Education](https://developers.google.com/edu/python/regular-expressions)  [\(https://developers.google.com/edu/python/regular-expressions\)](https://developers.google.com/edu/python/regular-expressions)